PRACTICAL-2

Name:Aarryan Parakh
Class:B4-B2-29
Branch:DSA

Functional and Non Functional Requirements

Requirements analysis is a crucial phase in software development where the needs and expectations of users and stakeholders are identified and documented. It ensures the system is built correctly and meets its intended goals.

- Functional Requirements: Define what the system should do (features and operations).
- Non-Functional Requirements: Define how the system should perform (quality, performance, and constraints).

Functional Requirements For Our Project

Title: Autonomous Codebase Onboarding Agent

Functional Requirements

These describe what the system should do.

1. **Repository Integration**
    - Connect to Git repositories (GitHub, GitLab, Bitbucket).
    - Clone or access repositories using secure authentication.

2. **Codebase Analysis**
    - Scan the project structure automatically.
    - Detect programming languages, frameworks, and libraries used.
    - Identify important files such as README, configuration files, and dependency files.

3. **Dependency Extraction**
    - Read package/dependency files (e.g., package.json, requirements.txt, pom.xml).
    - Generate a list of project dependencies and versions.

4. **Architecture Detection**
    - Analyze folder hierarchy and module relationships.

    - Identify design patterns and software architecture (MVC, Microservices, Layered Architecture, etc.).
5. **Documentation Generation**
    - Generate summaries of project modules.
    - Create onboarding documentation for new developers.
    - Explain the purpose of key files and directories.

6. **Code Explanation**
    - Provide natural language explanations for classes, methods, and functions.
    - Answer developer questions about the codebase using AI.

7. **Dependency Graph Visualization**
    - Display relationships between modules, packages, and files.
    - Generate architecture or flow diagrams.

8. **Search and Query Interface**
    - Allow users to search files, functions, classes, and APIs.
    - Support natural language queries like:
        - "Where is user authentication implemented?"
        - "Which module handles database connections?"

9. **Knowledge Base Creation**
    - Store analyzed project information for future queries.
    - Update the knowledge base whenever the repository changes.

10. **Version Tracking**

- Detect repository updates.
- Re-analyze modified files automatically.

11. **User Authentication**

- Secure login for users.
- Role-based access (Developer, Team Lead, Admin).

12. **Report Generation**

- Export onboarding reports in PDF or HTML.
- Include architecture overview, dependencies, and project summary.

Non-Functional Requirements

These describe how well the system performs.

1. Performance
- Analyze medium-sized repositories within 2–5 minutes.
- Respond to user queries within 3–5 seconds.

2. Scalability
- Support repositories ranging from small projects to enterprise-scale codebases.
- Handle multiple users simultaneously.

3. Reliability
- Ensure consistent analysis results.
- Recover gracefully from interrupted scans or network failures.

4. Security
- Encrypt repository access tokens and credentials.
- Implement secure authentication and authorization.
- Protect sensitive source code from unauthorized access.

5. Availability
- Maintain high system uptime (e.g., 99.9% availability).
- Support continuous monitoring and recovery.

6. Usability
- Provide an intuitive dashboard.
- Offer clear navigation and easy-to-understand visualizations.
- Support developers with minimal training.

7. Maintainability
- Use a modular architecture for easy updates.
- Allow new language analyzers and AI models to be added with minimal changes.

8. Compatibility
- Support major operating systems (Windows, Linux, macOS).
- Work with popular Git platforms and IDEs.

9. Accuracy
- Correctly identify project components and dependencies.
- Generate reliable AI-based explanations with minimal hallucinations.

10. Extensibility
- Support additional programming languages and frameworks through plugins.
- Allow integration with CI/CD tools and project management platforms.

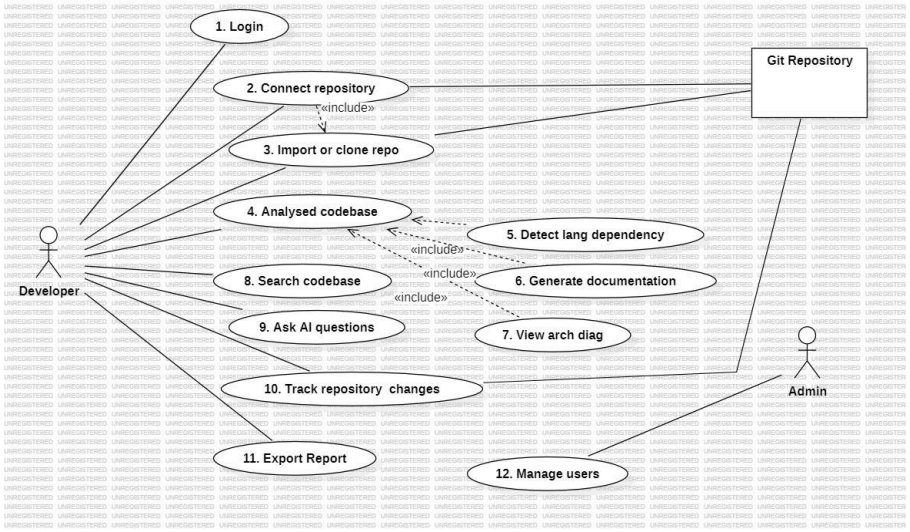
11. Portability

- Deploy on cloud platforms or on-premises environments.
- Support containerized deployment using Docker.

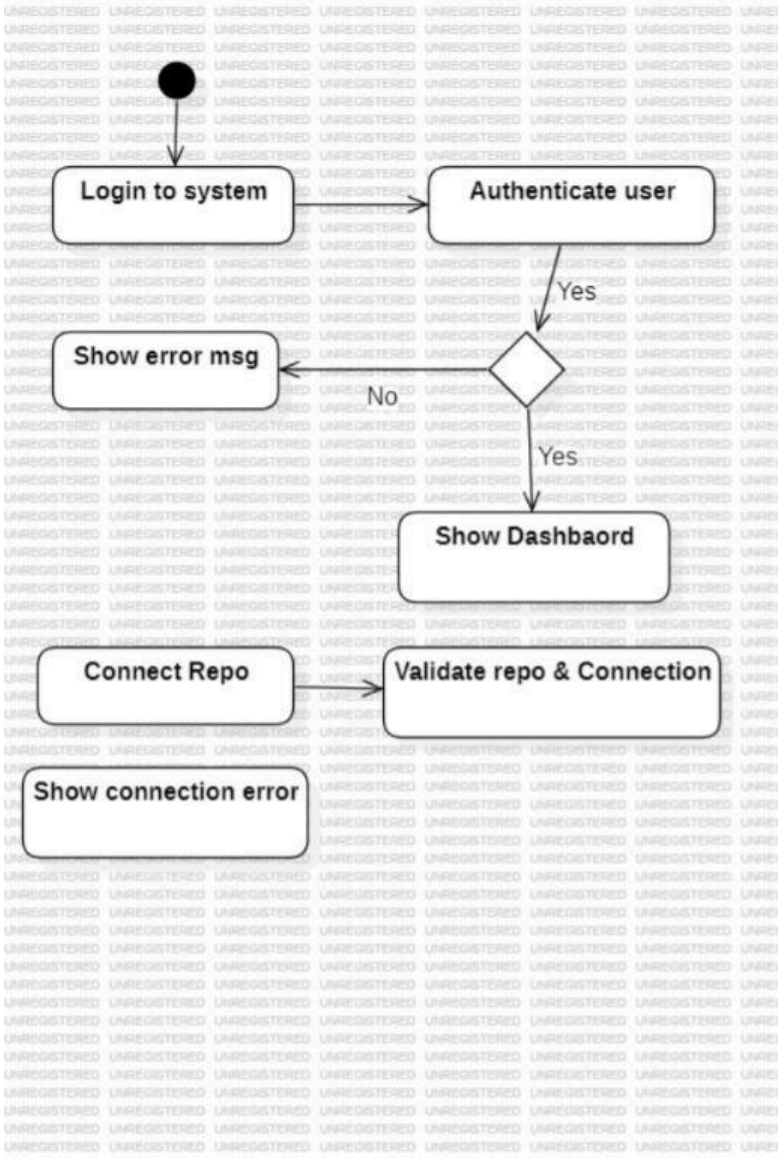
12. Logging and Monitoring

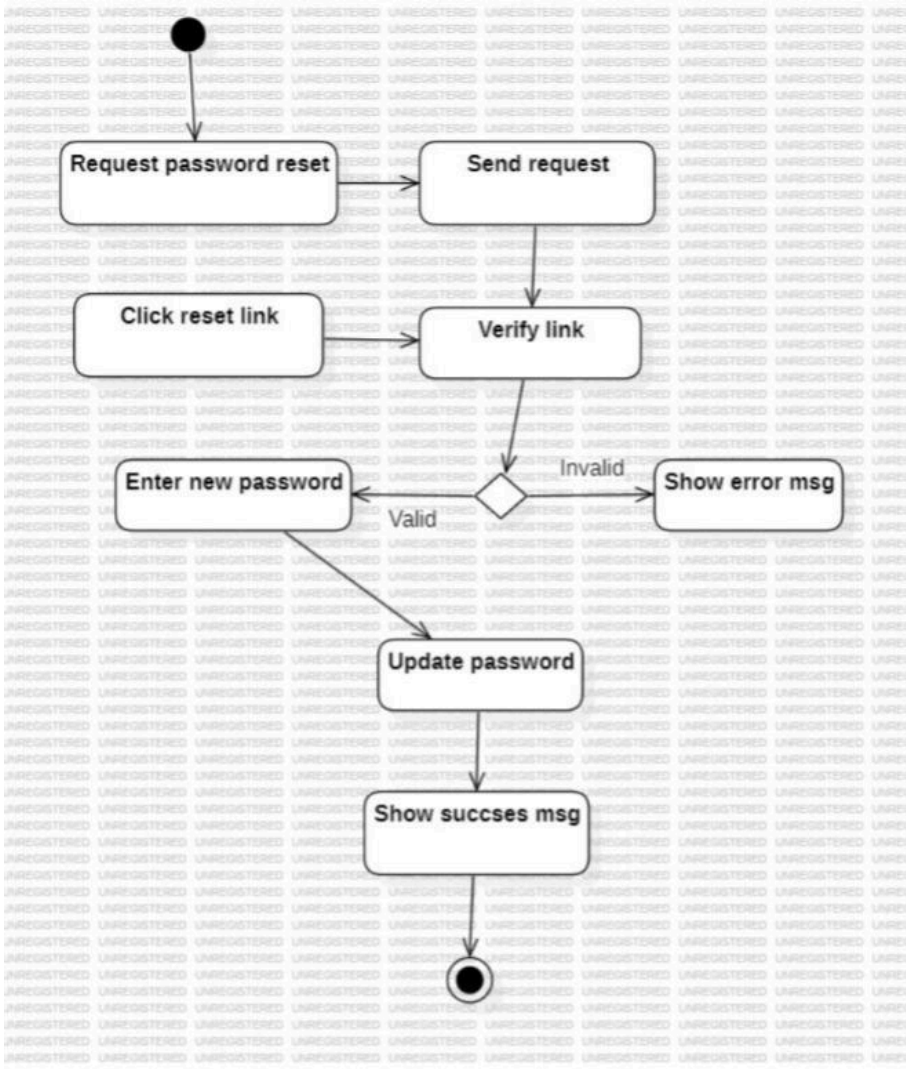
- Maintain logs of repository analysis and user activities.
- Monitor system performance and detect failures automatically.

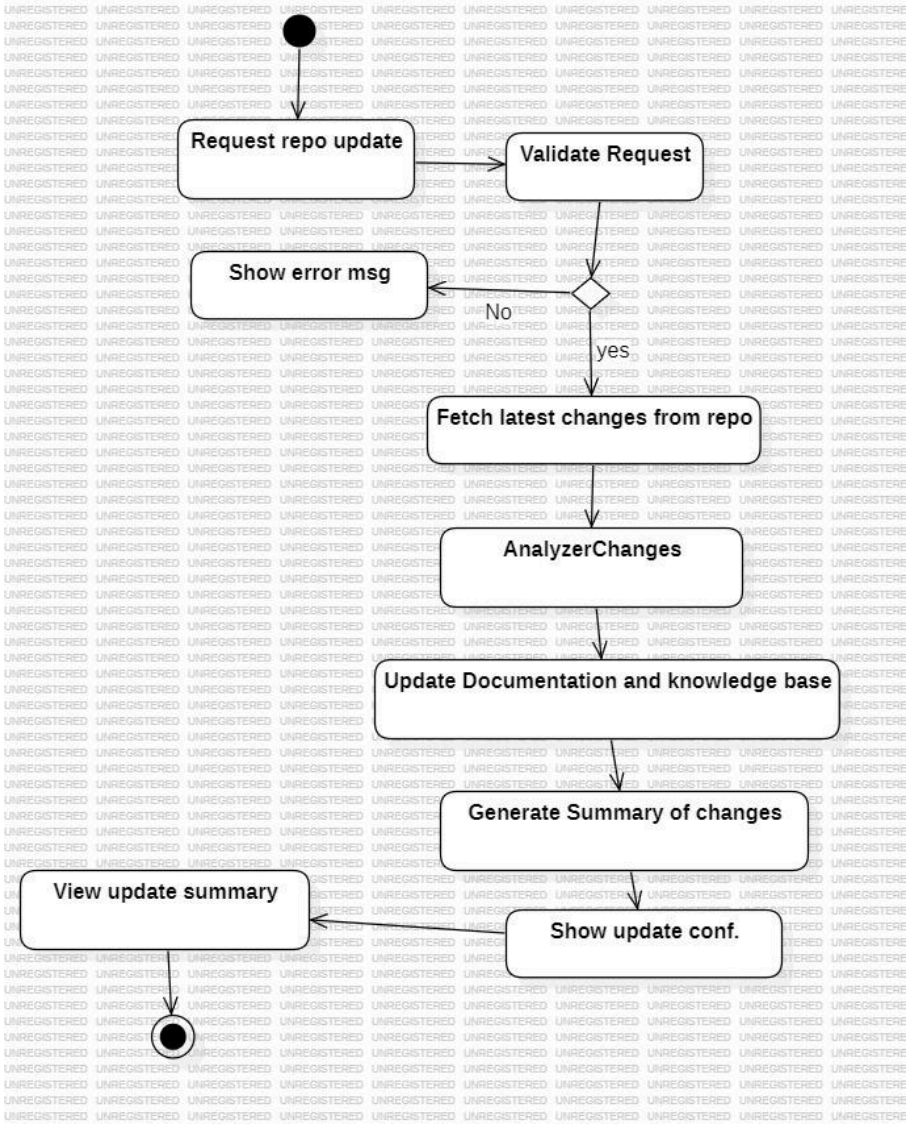
Use case diagram on the next page



Activity diagram for our problem statement :


Login to system
Authenticate user
Yes
Show error msg
No
Yes
Show Dashbaord
Connect Repo
Validate repo & Connection
Show connection error


Request password reset
Send request
Click reset link
Verify link
Enter new password
Invalid
Show error msg
Valid
Update password
Show succses msg


Request repo update
Validate Request
Show error msg
No
yes
Fetch latest changes from repo
AnalyzerChanges
Update Documentation and knowledge base
Generate Summary of changes
View update summary
Show update conf.